NETWORK PORT SCANNER

KOTHURI SHIVA KUMAR

AICTE ID :STU683ecb06dcc261748945670

Problem Statement

In modern computer networks, identifying open and vulnerable ports is essential for maintaining security, yet many systems lack simple and effective tools for continuous monitoring. Unauthorized open ports can expose critical services to cyber threats such as hacking, malware, and data breaches. The challenge is to develop a network port scanner that can efficiently detect open, closed, and filtered ports across devices, provide accurate and timely results, and present the information in a user-friendly manner. This tool should help system administrators identify potential vulnerabilities, improve network security, and ensure proper configuration of services while minimizing false positives and scan time.

Project Description

A network port scanner is a cybersecurity tool used to probe a computer or network device to identify which ports are open, closed, or filtered, helping determine what services are running and potentially accessible. By sending packets to specific port numbers and analyzing the responses, a port scanner can map the network's exposure and detect vulnerabilities that attackers might exploit. System administrators and security professionals commonly use port scanners for auditing, troubleshooting, and strengthening defenses, while malicious actors may also use them to find entry points. Popular tools like Nmap automate this process and provide detailed insights into network configurations, making port scanning an essential practice in both offensive and defensive security.

WHO ARE THE END USERS?

The end users of a network port scanner include system administrators, network engineers, cybersecurity professionals, and ethical hackers, who use it to monitor, manage, and secure networks. Additionally, IT support teams and organizations use port scanners to identify vulnerabilities, troubleshoot connectivity issues, and ensure proper network configuration.

Technology Used

- ChatGPT
- Google Gemini

RESULTS

```python
import socket
import threading
import time
import queue
import sys
import tkinter as tk
from tkinter import ttk, messagebox, filedialog

# ----------------------------
# Service Map (extend freely)
# ----------------------------
COMMON_PORTS = {
    21: 'FTP', 22: 'SSH', 23: 'Telnet', 25: 'SMTP', 53: 'DNS',
    80: 'HTTP', 110: 'POP3', 143: 'IMAP', 443: 'HTTPS',
    3306: 'MySQL', 3389: 'RDP', 5900: 'VNC', 8080: 'HTTP-Alt'
}

# ----------------------------
# Scanner Worker
# ----------------------------
class PortScanner:
    def __init__(self, target, start_port, end_port, timeout=0.5, max_workers=500):
        self.target = target
        self.start_port = start_port
        self.end_port = end_port
        self.timeout = timeout          # internal default
        self.max_workers = max_workers  # internal default
        self._stop_event = threading.Event()

        self.total_ports = max(0, end_port - start_port + 1)
        self.scanned_count = 0
        self.open_ports = []            # list[(port, service)]
        self._lock = threading.Lock()
        self.result_queue = queue.Queue()

    def stop(self):
        self._stop_event.set()

    def _scan_port(self, port):
        if self._stop_event.is_set():
            return
        try:
            s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            s.settimeout(self.timeout)
            result = s.connect_ex((self.target, port))
            if result == 0:
                service = COMMON_PORTS.get(port, 'Unknown')
```

```python
                with self._lock:
                    self.open_ports.append((port, service))
                self.result_queue.put(('open', port, service))
            s.close()
        except Exception as e:
            self.result_queue.put(('error', port, str(e)))
        finally:
            with self._lock:
                self.scanned_count += 1
            self.result_queue.put(('progress', self.scanned_count, self.total_ports))

    def resolve_target(self):
        return socket.gethostbyname(self.target)

    def run(self):
        sem = threading.Semaphore(self.max_workers)
        threads = []

        for port in range(self.start_port, self.end_port + 1):
            if self._stop_event.is_set():
                break
            sem.acquire()
            t = threading.Thread(target=self._worker_wrapper, args=(sem, port), daemon=True)
            threads.append(t)
            t.start()

        for t in threads:
            t.join()

        self.result_queue.put(('done', None, None))

    def _worker_wrapper(self, sem, port):
        try:
            self._scan_port(port)
        finally:
            sem.release()

# ----------------------------
# Tkinter GUI (minimal inputs)
# ----------------------------
class ScannerGUI(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Network Port Scanner - Minimal GUI")
        self.geometry("720x520")
        self.minsize(680, 480)
```

Demo link

RESULTS

```python
    self.scanner_thread = None
    self.scanner = None
    self.start_time = None
    self.poll_after_ms = 40

    self._build_ui()

def _build_ui(self):
    # --- Top Frame: Inputs (Only 3 fields) ---
    frm_top = ttk.LabelFrame(self, text="Scan Settings")
    frm_top.pack(fill="x", padx=10, pady=10)

    ttk.Label(frm_top, text="Target (IP / Hostname):").grid(row=0, column=0, padx=8, pady=8, sticky="e")
    self.ent_target = ttk.Entry(frm_top, width=36)
    self.ent_target.grid(row=0, column=1, padx=8, pady=8, sticky="w")

    ttk.Label(frm_top, text="Start Port:").grid(row=0, column=2, padx=8, pady=8, sticky="e")
    self.ent_start = ttk.Entry(frm_top, width=10)
    self.ent_start.insert(0, "1")
    self.ent_start.grid(row=0, column=3, padx=8, pady=8, sticky="w")

    ttk.Label(frm_top, text="End Port:").grid(row=0, column=4, padx=8, pady=8, sticky="e")
    self.ent_end = ttk.Entry(frm_top, width=10)
    self.ent_end.insert(0, "1024")
    self.ent_end.grid(row=0, column=5, padx=8, pady=8, sticky="w")

    self.btn_start = ttk.Button(frm_top, text="Start Scan", command=self.start_scan)
    self.btn_start.grid(row=1, column=4, padx=8, pady=8, sticky="e")

    self.btn_stop = ttk.Button(frm_top, text="Stop", command=self.stop_scan, state="disabled")
    self.btn_stop.grid(row=1, column=5, padx=8, pady=8, sticky="w")

    for i in range(6):
        frm_top.grid_columnconfigure(i, weight=1)

    # --- Progress / Status ---
    frm_status = ttk.LabelFrame(self, text="Status")
    frm_status.pack(fill="x", padx=10, pady=(0,10))

    self.var_status = tk.StringVar(value="Idle")
    self.lbl_status = ttk.Label(frm_status, textvariable=self.var_status)
    self.lbl_status.pack(side="left", padx=10, pady=8)

    self.var_elapsed = tk.StringVar(value="Elapsed: 0.00s")
    self.lbl_elapsed = ttk.Label(frm_status, textvariable=self.var_elapsed)
    self.lbl_elapsed.pack(side="right", padx=10, pady=8)
```

```python
    self.progress = ttk.Progressbar(frm_status, orient="horizontal", mode="determinate")
    self.progress.pack(fill="x", padx=10, pady=(0,10))

    # --- Results ---
    frm_results = ttk.LabelFrame(self, text="Open Ports")
    frm_results.pack(fill="both", expand=True, padx=10, pady=(0,10))

    self.txt_results = tk.Text(frm_results, height=16, wrap="none")
    self.txt_results.pack(fill="both", expand=True, side="left", padx=(10,0), pady=10)

    yscroll = ttk.Scrollbar(frm_results, orient="vertical", command=self.txt_results.yview)
    yscroll.pack(side="right", fill="y", pady=10)
    self.txt_results.configure(yscrollcommand=yscroll.set)

    xscroll = ttk.Scrollbar(self, orient="horizontal", command=self.txt_results.xview)
    xscroll.pack(fill="x", padx=10, pady=(0,10))
    self.txt_results.configure(xscrollcommand=xscroll.set)

    # --- Bottom Buttons ---
    frm_bottom = ttk.Frame(self)
    frm_bottom.pack(fill="x", padx=10, pady=(0,12))

    self.btn_clear = ttk.Button(frm_bottom, text="Clear", command=self.clear_results)
    self.btn_clear.pack(side="left")

    self.btn_save = ttk.Button(frm_bottom, text="Save Results", command=self.save_results, state="disabled")
    self.btn_save.pack(side="right")

# -----------------------
# Control Handlers
# -----------------------
def start_scan(self):
    if self.scanner_thread and self.scanner_thread.is_alive():
        messagebox.showinfo("Scanner", "A scan is already running.")
        return

    target = self.ent_target.get().strip()
    if not target:
        messagebox.showerror("Input Error", "Please enter a target IP or hostname.")
        return

    try:
        start_port = int(self.ent_start.get().strip())
        end_port = int(self.ent_end.get().strip())
    except ValueError:
        messagebox.showerror("Input Error", "Ports must be integers.")
        return
```

Demo link

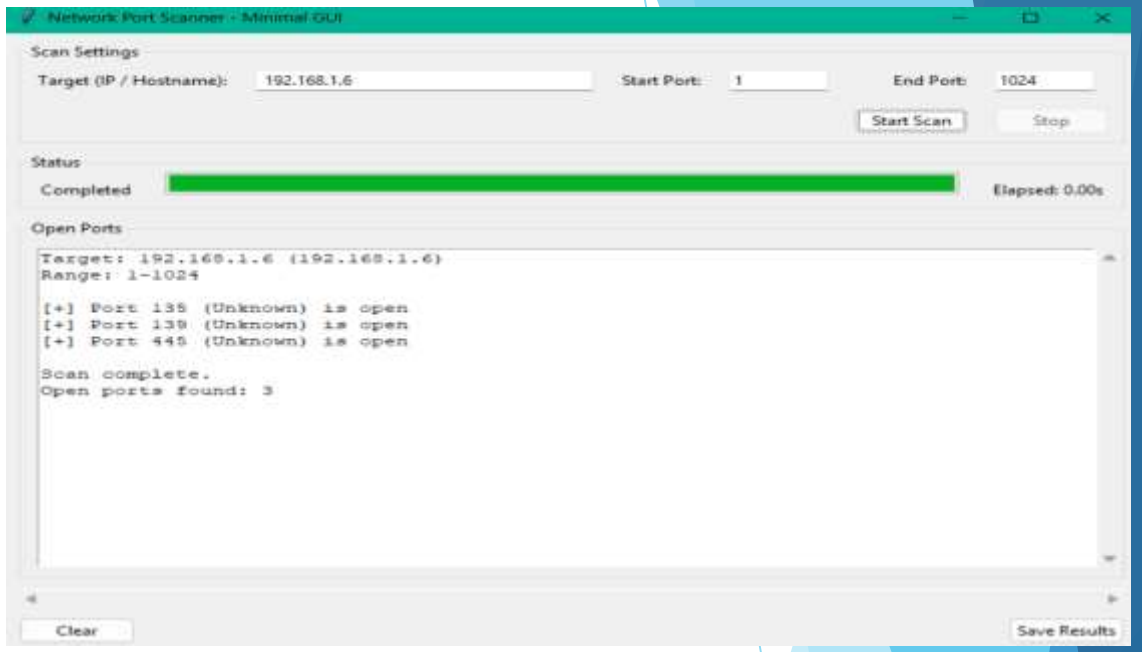
RESULTS

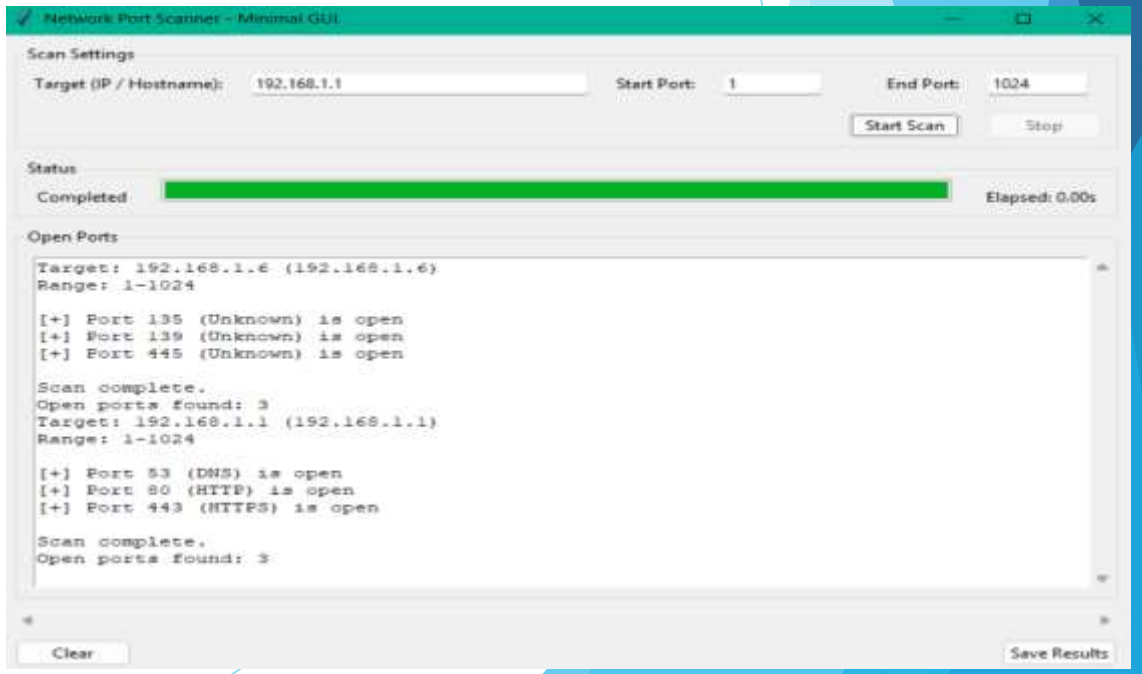
```python
    if not (0 <= start_port <= 65535 and 0 <= end_port <= 65535 and start_port <= end_port):
        messagebox.showerror("Input Error", "Port range must be within 0-65535 and start ≤ end.")
        return

    # Internal defaults (not shown in UI)
    timeout = 0.5
    max_threads = 500

    self.scanner = PortScanner(target, start_port, end_port, timeout=timeout, max_workers=max_threads)

    # Pre-resolve target to catch DNS issues early
    try:
        resolved_ip = self.scanner.resolve_target()
        self.append_text(f"Target: {target} ({resolved_ip})\n")
        self.append_text(f"Range: {start_port}-{end_port}\n\n")
    except Exception as e:
        messagebox.showerror("Resolution Error", f"Failed to resolve target '{target}'.\n{e}")
        self.scanner = None
        return

    self.btn_start.configure(state="disabled")
    self.btn_stop.configure(state="normal")
    self.btn_save.configure(state="disabled")
    self.clear_progress()

    self.start_time = time.time()
    self.var_status.set("Scanning...")
    self.update_elapsed()

    self.scanner_thread = threading.Thread(target=self.scanner.run, daemon=True)
    self.scanner_thread.start()

    self.after(self.poll_after_ms, self.poll_results)

def stop_scan(self):
    if self.scanner:
        self.scanner.stop()
        self.var_status.set("Stopping...")

def clear_results(self):
    self.txt_results.delete("1.0", tk.END)
    self.clear_progress()
    self.var_status.set("Idle")
    self.var_elapsed.set("Elapsed: 0.00s")
    self.btn_save.configure(state="disabled")
def save_results(self):
    if not self.scanner or not self.scanner.open_ports:
        messagebox.showinfo("Save Results", "No open ports to save.")
        return
    default_name = f"open_ports_{int(time.time())}.txt"
    file_path = filedialog.asksaveasfilename(
        title="Save results",
        defaultextension=".txt",
        initialfile=default_name,
        filetypes=[("Text Files", "*.txt"), ("All Files", "*.*")]
    )
    if not file_path:
        return
    try:
        with open(file_path, "w", encoding="utf-8") as f:
            f.write("Open Ports:\n")
            for port, service in sorted(self.scanner.open_ports, key=lambda x: x[0]):
                f.write(f"Port {port} ({service}) is open\n")
        messagebox.showinfo("Saved", f"Results saved to:\n{file_path}")
    except Exception as e:
        messagebox.showerror("Save Error", f"Failed to save file.\n{e}")
# ------------------------
# UI Helpers
# ------------------------
def append_text(self, text):
    self.txt_results.insert(tk.END, text)
    self.txt_results.see(tk.END)

def clear_progress(self):
    self.progress.configure(value=0, maximum=1)

def update_elapsed(self):
    if self.start_time and self.var_status.get() in ("Scanning...", "Stopping..."):
        elapsed = time.time() - self.start_time
        self.var_elapsed.set(f"Elapsed: {elapsed:.2f}s")
        self.after(200, self.update_elapsed)

def poll_results(self):
    if not self.scanner:
        return
    try:
        while True:
            msg_type, a, b = self.scanner.result_queue.get_nowait()
            if msg_type == 'open':
                port, service = a, b
                self.append_text(f"[+] Port {port} ({service}) is open\n")
            elif msg_type == 'progress':
                scanned, total = a, b
                self.progress.configure(maximum=max(total, 1), value=scanned)
```

RESULTS

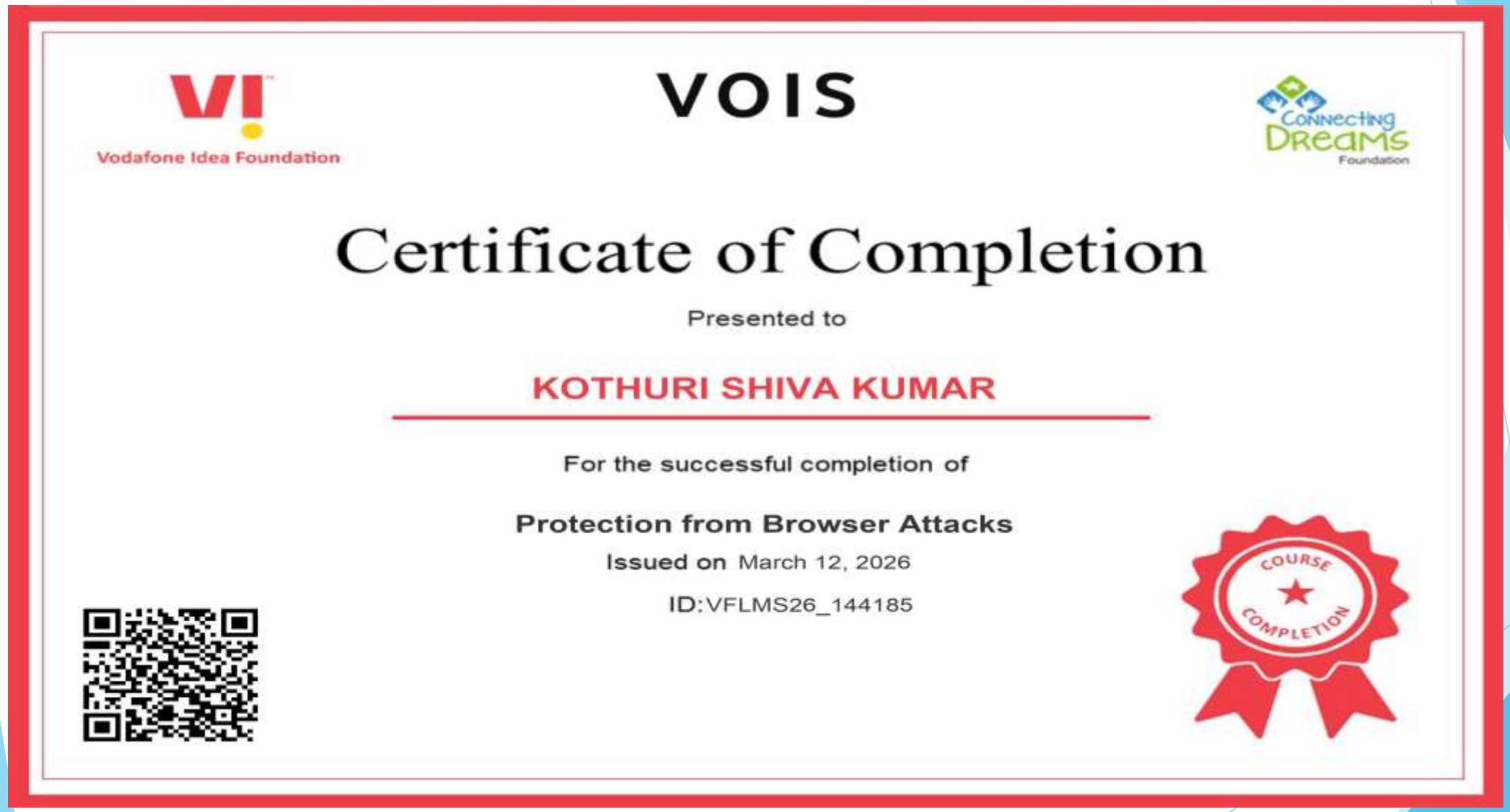
```python
            while True:
                msg_type, a, b = self.scanner.result_queue.get_nowait()
                if msg_type == 'open':
                    port, service = a, b
                    self.append_text(f"[+] Port {port} ({service}) is open\n")
                elif msg_type == 'progress':
                    scanned, total = a, b
                    self.progress.configure(maximum=max(total, 1), value=scanned)
                    self.var_status.set(f"Scanning... {scanned}/{total}")
                elif msg_type == 'done':
                    total_open = len(self.scanner.open_ports)
                    self.append_text("\nScan complete.\n")
                    self.append_text(f"Open ports found: {total_open}\n")
                    self.var_status.set("Completed")
                    self.btn_start.configure(state="normal")
                    self.btn_stop.configure(state="disabled")
                    self.btn_save.configure(state="normal" if total_open else "disabled")
                    self.start_time = None
        except queue.Empty:
            pass

        if self.scanner_thread and self.scanner_thread.is_alive():
            self.after(self.poll_after_ms, self.poll_results)
        else:
            if self.var_status.get() in ("Scanning...", "Stopping..."):
                self.var_status.set("Completed")
            self.btn_start.configure(state="normal")
            self.btn_stop.configure(state="disabled")
            if self.scanner and self.scanner.open_ports:
                self.btn_save.configure(state="normal")

def main():
    # Windows console nicety if launched from terminal
    if sys.platform.startswith("win"):
        try:
            import ctypes
            kernel32 = ctypes.windll.kernel32
            kernel32.SetConsoleMode(kernel32.GetStdHandle(-10), 7)
        except Exception:
            pass

    app = ScannerGUI()
    app.mainloop()

if __name__ == "__main__":
    main()
```

OUTPUT:





Certificate - Protection from browser attacks



Vodafone Idea Foundation

VOIS

Connecting Dreams Foundation

Certificate of Completion

Presented to

KOTHURI SHIVA KUMAR

For the successful completion of

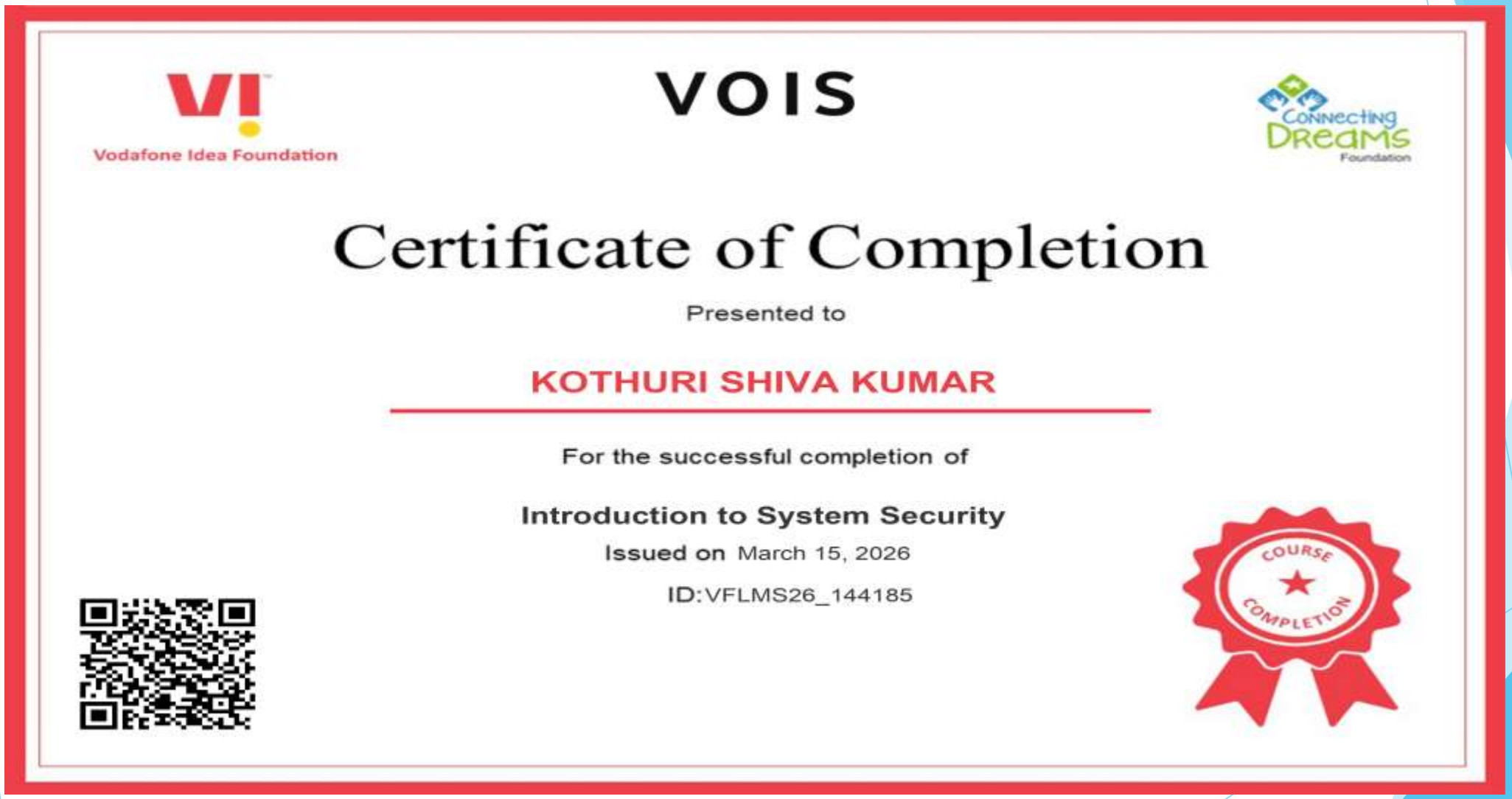
Protection from Browser Attacks

Issued on March 12, 2026

ID:VFLMS26_144185

COURSE COMPLETION

Certificate - Introduction to System Security



Vodafone Idea Foundation

VOIS

Connecting Dreams Foundation

Certificate of Completion

Presented to

KOTHURI SHIVA KUMAR

For the successful completion of

Introduction to System Security

Issued on March 15, 2026

ID:VFLMS26_144185

COURSE COMPLETION

GitHub link

Click This Logo To Get Project Link :

THANK YOU